

System Design: Event-Ops AI Agentic RAG

1. Executive Summary

The **Event-Ops AI Agent** is a specialized Agentic RAG application designed to bridge the communication gap between vendors and on-ground event teams. By indexing event-brief documents into a high-performance vector store (Endee), the system provides real-time decision support, solves logistical issues, and suggests actionable plans based on ground realities.

2. High-Level Architecture

The system follows a modular **Agent-over-RAG** pattern. Instead of a static retrieval pipeline, an autonomous agent orchestrates the flow based on the complexity of the user's operational query.

2.1 Component Overview

1. **Ingestion Engine:** Parses event briefs (PDF/Docx) and chunks text for indexing.
2. **Endee Vector DB:** Stores high-dimensional embeddings of the event data for semantic retrieval.
3. **Agentic Orchestrator:** Uses an LLM (Gemini 1.5 Flash) to reason through issues and formulate plans.
4. **Action Tools:** Specialized functions the agent can call (e.g., `get_vendor_contact`, `suggest_pivot_plan`).
5. **Interface:** A lightweight frontend for on-ground personnel to report issues.

3. Detailed Data Flow

Step A: Document Ingestion (The "Briefing" Phase)

- **Source:** Event Briefing Documents.
- **Processing:** Documents are split into semantic chunks (e.g., "Vendor Info," "Timeline," "Crisis Protocol").
- **Embedding:** Chunks are converted to vectors using a standard embedding model.
- **Storage:** Vectors are indexed in the **Endee Repository** fork.

Step B: The Agentic Loop (The "Operational" Phase)

When an on-ground user reports an issue (e.g., *"The catering truck is stuck in traffic, what's the plan?"*):

1. **Intent Analysis:** The Agent determines if this is a simple "Lookup" (Who is the caterer?) or a "Decision" task (How to handle the delay?).
2. **Context Retrieval:** The Agent queries **Endee** to find the catering contract, the event timeline, and backup food options.
3. **Reasoning & Planning:** The Agent compares the "Delay" with the "Event Timeline" found in the brief.
4. **Response Generation:** The Agent outputs a decision: *"Contact [Name] at [Phone]. Move the appetizers to 6:30 PM to buy 30 mins of buffer."*

4. Technical Stack

Layer	Technology	Reason
Orchestration	Python (FastAPI)	High performance for async AI calls.
Vector Store	Endee (Official Fork)	High-speed indexing and mandated repository.
LLM	Gemini 1.5 Flash	Low latency and large context window for long briefs.
Embeddings	OpenAI/Gemini Embeddings	State-of-the-art semantic representation.
Front-End	Streamlit (Optional)	Rapid prototyping for on-ground testing.

5. Security & Constraints

- **Data Isolation:** Each event's data is stored in a unique collection/index within Endee to prevent cross-contamination.
- **Safety Guardrails:** The agent is restricted from making financial commitments; it only *suggests* plans for human approval.
- **Cost Efficiency:** Using "Flash" models and efficient vector search minimizes per-query costs.

6. Future Scalability (Post-Testing)

- **Multi-Agent Systems:** One agent for "Logistics" and one for "Vendor Relations" communicating via a shared blackboard.

- **Real-time Telemetry:** Integrating with GPS/IoT to automatically trigger "Issue Solving" alerts when a vendor is physically late.

7. Compliance Checklist

- [x] Starred official Endee repository.
- [x] Forked repository used as the base.
- [x] RAG system implementation for event briefs.